MCP Server Tool Reference

flowise-questionnaire-system

Generated 2026-08-17

Every tool below is documented with its real Python signature, its matching MCP prompt (slash-command shortcut), and a complete, real request/response example -- generated by actually calling each tool function against a small, deterministic fixture (the same benefits_w18 example used throughout this project's test suite and skill documentation), not hand-written or fabricated. Server is read-only: it never writes to the database and never triggers expensive pipeline steps (extraction, graph building, matching, AI review) itself -- every tool reads data some other part of the app already computed and persisted.

Running the server

python manage.py runmcp -- its own standalone process, own port (8765 by default via $MCP_SERVER_HOST/$MCP_SERVER_PORT), own streamable-HTTP transport. Does not touch config/asgi.py, wsgi.py, or manage.py runserver. HTTP-only by design (no stdio transport). Serves HTTPS via a local self-signed cert in mcp_server/certs/ if present when run locally; in production, nginx terminates TLS and runmcp runs behind it as plain HTTP (--no-tls).

Authentication

Per-person token embedded in the URL path: https://<host>/t/<token>/mcp (not a header -- MCP client connector UIs generally only take a URL, e.g. Claude Desktop's "Add custom connector"). Admins generate/revoke tokens at /questionnaires/mcp-tokens/ (staff-only page). A bare /mcp (no token) always 404s; an unknown/revoked token 403s; revocation takes effect immediately (a live DB check, not a config reload).

Colectica / Forsta+ scope, at a glance

Most tools are Colectica-only and reject Forsta+ modules with a clear ToolError. Two tools (get_module_graph, evaluate_edge_condition) support either format on a single module. Two pairs of tools compare two modules against each other: the Colectica-vs-Forsta+ pair (get_routing_diff_report / get_routing_discrepancy_detail) requires one module of each format; the Colectica-vs-Colectica "version diff" pair (get_version_diff_report / get_version_diff_discrepancy_detail) requires both modules to be Colectica JSON -- e.g. two waves/versions of the same questionnaire. These are two independent comparators with different matching priorities and discrepancy types, not the same code reused.

Contents

CORE COLECTICA TOOLS

list_modules

Scope: Colectica-only

MCP prompt: /listModules

Description

List every Colectica JSON QuestionnaireModule: id, name, status, question_count, edge_count.

Python signature

```
list_modules(

) -> dict
```

Example prompt shortcut

```
/listModules

"Using the list_modules tool, list every Colectica questionnaire module available, with its status, question count, and routing-edge count."
```

Example request

```
list_modules()
```

Example response (real output)


```
{
  "count": 8,
  "modules": [
    {
      "id": "b874d0c0-4731-4502-887d-02dfa7c70a30",
      "name": "benefits_w19",
      "version": "v2",
      "status": "extracted",
      "question_count": 1,
      "edge_count": 1
    },
    {
      "id": "e337e498-34ba-47d2-a06f-c8f2c4e253be",
      "name": "benefits_w18",
      "version": "v1",
      "status": "extracted",
      "question_count": 5,
      "edge_count": 4
    },
    {
      "id": "082097ea-506f-42bc-b93e-df24afe88190",
      "name": "W16",
      "version": "v02",
      "status": "extracted",
      "question_count": 1981,
      "edge_count": 4465
    },
    {
      "id": "234f2404-7ed7-476a-afce-7e2dbe5d12e3",
      "name": "colectica_test_2",
      "version": "02",
      "status": "extracted",
```

```
      "question_count": 1803,
      "edge_count": 4152
    },
    {
      "id": "2ab9692f-5f5f-4d88-a714-002cac97c977",
      "name": "W18",
      "version": "v01",
      "status": "uploaded",
      "question_count": 0,
      "edge_count": 0
    },
    {
      "id": "3a2de4bf-432b-42f2-b74c-2ceffd868e8a",
      "name": "W18",
      "version": "v01",
      "status": "extracted",
      "question_count": 1803,
      "edge_count": 4152
    },
    {
      "id": "c4fe3c04-1cda-40d7-9028-a43988c4c9dc",
      "name": "Full Wave 18 Questionnaire",
      "version": "v1",
      "status": "uploaded",
      "question_count": 0,
      "edge_count": 0
    },
    {
      "id": "c8171847-7442-41ef-b534-04682184a14f",
      "name": "Benefits W18",
      "version": "v02",
      "status": "extracted",
      "question_count": 20,
      "edge_count": 35
    }
  ]
}
```

get_module_summary

Scope: Colectica-only

MCP prompt: /showModuleSummary

Description

Full detail on one Colectica module: status, counts, timestamps.

Raises a clear ToolError (not an empty result) if the module exists but is a Forsta+ XML module.

Python signature

```
get_module_summary(
    module_id_or_name: str
) -> dict
```

Example prompt shortcut

```
/showModuleSummary

"Using the get_module_summary tool, give me the full status summary for module 'benefits_w18' (question/edge counts by type, whether a graph has been built, and any error message)."
```

Example request

```
get_module_summary(module_id_or_name="benefits_w18")
```

Example response (real output)


```
{
  "id": "e337e498-34ba-47d2-a06f-c8f2c4e253be",
  "name": "benefits_w18",
  "version": "v1",
  "status": "extracted",
  "error_message": "",
  "question_count": 5,
  "routing_edge_count": 4,
  "conditional_edge_count": 3,
  "loop_edge_count": 0,
  "sequential_edge_count": 1,
  "has_graph": true,
  "created_at": "2026-08-17T01:58:07.199821+00:00",
  "updated_at": "2026-08-17T01:58:07.199821+00:00"
}
```

list_questions

Scope: Colectica-only

MCP prompt: /listQuestions

Description

All NormalizedQuestion rows for a Colectica module (name, text, type, options).

Python signature

```
list_questions(
    module_id_or_name: str
) -> dict
```

Example prompt shortcut

```
/listQuestions

"Using the list_questions tool, list every question in module 'benefits_w18' with its type and options."
```

Example request

```
list_questions(module_id_or_name="benefits_w18")
```

Example response (real output)


```
{
  "module_id": "e337e498-34ba-47d2-a06f-c8f2c4e253be",
  "module_name": "benefits_w18",
  "count": 5,
  "questions": [
    {
      "name": "AIDHH",
      "label": "Any dependent household member",
      "text": "Does anyone in your household provide regular care?",
      "question_type": "single_choice",
      "options": [
        {
          "code": "1",
          "label": "Yes"
        },
        {
          "code": "2",
          "label": "No"
        }
      ],
      "help_text": "",
      "interviewer_instruction": "",
      "sequence_index": 1
    },
    {
      "name": "NAIDHH",
      "label": "Number of carers",
      "text": "How many people provide care?",
      "question_type": "number",
      "options": [],
      "help_text": "",
      "interviewer_instruction": "",
      "sequence_index": 2
    },
    {
```

```
      "name": "BENBASE",
      "label": "Benefit base question",
      "text": "Which benefits does your household receive?",
      "question_type": "multi_choice",
      "options": [
        {
          "code": "1",
          "label": "Universal Credit"
        },
        {
          "code": "2",
          "label": "Housing Benefit"
        },
        {
          "code": "3",
          "label": "Other"
        }
      ],
      "help_text": "",
      "interviewer_instruction": "",
      "sequence_index": 3
    },
    {
      "name": "FICODE",
      "label": "Main income source",
      "text": "What is the main source of family income?",
      "question_type": "single_choice",
      "options": [
        {
          "code": "1",
          "label": "Employment"
        },
        {
          "code": "2",
          "label": "Benefits"
        }
      ],
      "help_text": "",
      "interviewer_instruction": "",
      "sequence_index": 4
    },
    {
      "name": "FRVAL",
      "label": "Weekly income value",
      "text": "Value of that income per week?",
      "question_type": "number",
      "options": [],
      "help_text": "",
      "interviewer_instruction": "",
      "sequence_index": 5
    }
  ]
}
```

get_question

Scope: Colectica-only

MCP prompt: /showQuestion

Description

Single question lookup, case-insensitive on name.

Python signature

```
get_question(
    module_id_or_name: str,
    question_name: str
) -> dict
```

Example prompt shortcut

```
/showQuestion

"Using the get_question tool, show me the full detail for question 'AIDHH' in module 'benefits_w18'."
```

Example request

```
get_question(module_id_or_name="benefits_w18", question_name="AIDHH")
```

Example response (real output)


```
{
  "module_id": "e337e498-34ba-47d2-a06f-c8f2c4e253be",
  "module_name": "benefits_w18",
  "name": "AIDHH",
  "label": "Any dependent household member",
  "text": "Does anyone in your household provide regular care?",
  "question_type": "single_choice",
  "options": [
    {
      "code": "1",
      "label": "Yes"
    },
    {
      "code": "2",
      "label": "No"
    }
  ],
  "help_text": "",
  "interviewer_instruction": "",
  "sequence_index": 1
}
```

get_routing_edges

Scope: Colectica-only

MCP prompt: /listRoutingEdges

Description

All RoutingEdge rows for a Colectica module (source, target, condition, type).

Python signature

```
get_routing_edges(
    module_id_or_name: str
) -> dict
```

Example prompt shortcut

```
/listRoutingEdges

"Using the get_routing_edges tool, list every routing edge in module 'benefits_w18' (source, target, condition, edge type)."
```

Example request

```
get_routing_edges(module_id_or_name="benefits_w18")
```

Example response (real output)


```
{
  "module_id": "e337e498-34ba-47d2-a06f-c8f2c4e253be",
  "module_name": "benefits_w18",
  "count": 4,
  "edges": [
    {
      "source_question": "AIDHH",
      "target_question": "NAIDHH",
      "condition_text": "if [AIDHH = 1]",
      "edge_type": "conditional",
      "sequence_index": 1
    },
    {
      "source_question": "NAIDHH",
      "target_question": "BENBASE",
      "condition_text": "if [NAIDHH > 1]",
      "edge_type": "conditional",
      "sequence_index": 2
    },
    {
      "source_question": "HHGRID.HHSIZE",
      "target_question": "FICODE",
      "condition_text": "if [(HHGRID.HHSIZE - absent members) > 1]",
      "edge_type": "conditional",
      "sequence_index": 3
    },
    {
      "source_question": "FICODE",
      "target_question": "FRVAL",
      "condition_text": "",
      "edge_type": "sequential",
      "sequence_index": 4
    }
  ]
}
```

trace_variable

Scope: Colectica-only

MCP prompt: /traceVariable

Description

Find every RoutingEdge whose condition references this variable, across one Colectica module or all Colectica modules if module_id_or_name is omitted.

Uses the same external-variable rule as agentflow_payload_builder._extract_external_variables: a referenced variable is "external" if it is not the name of any NormalizedQuestion in that module and does not end in "Loop" (loop containers aren't external variables).

Python signature

```
trace_variable(
    variable_name: str,
    module_id_or_name: str | None = None
) -> dict
```

Example prompt shortcut

```
/traceVariable

"Using the trace_variable tool, find every routing edge referencing variable 'HHSIZE' across module 'benefits_w18', and tell me which references are external variables."
```

Example request

```
trace_variable(variable_name="HHSIZE", module_id_or_name="benefits_w18")
```

Example response (real output)


```
{
  "variable_name": "HHSIZE",
  "match_count": 1,
  "matches": [
    {
      "module_id": "e337e498-34ba-47d2-a06f-c8f2c4e253be",
      "module_name": "benefits_w18",
      "source_question": "HHGRID.HHSIZE",
      "target_question": "FICODE",
      "condition_text": "if [(HHGRID.HHSIZE - absent members) > 1]",
      "edge_type": "conditional",
      "sequence_index": 3,
      "is_external_variable": null
    }
  ]
}
```

CROSS-FORMAT TOOLS (COLECTICA + FORSTA+)

get_module_graph

Scope: Colectica + Forsta+

MCP prompt: /showModuleGraph

Description

Enriched routing graph for a module -- Colectica OR Forsta+, both are supported. Returns the same incoming_routes/outgoing_routes/ prerequisite_questions/start_semantics per node, and graph_start_summary overall, that the "View Graph" GUI page shows. Never builds the graph itself (that's a write path) -- raises ToolError if none has been built yet for this module.

Large modules cannot be returned in full: pass question_name to scope the response to that question's neighborhood (depth hops out, default 1, max 3) -- this is the primary way to explore a large module. Omit question_name only on modules with 50 or fewer questions ("mode": "full" in the response); on larger modules, omitting it returns "mode": "summary" (counts + graph_start_summary only, no per-node detail) rather than risk a huge response -- pass question_name in that case.

"mermaid" is Mermaid flowchart syntax (```mermaid fenced code renders as an actual diagram in most MCP clients, including Claude) for whatever is actually being returned -- the full graph in "full" mode, the scoped subgraph in "neighborhood" mode. Always None in "summary" mode, for the same size reason "graph" is None there.

Python signature

```
get_module_graph(
    module_id_or_name: str,
    question_name: str | None = None,
    depth: int = 1
) -> dict
```

Example prompt shortcut

```
/showModuleGraph

"Using the get_module_graph tool, show me the routing graph for module 'benefits_w18'. If it's too large to return in full, summarize graph_start_summary and the node/edge counts, then ask me which question to zoom into. If a "mermaid" field is returned, render it as a ```mermaid fenced code block."
```

Example request

```
get_module_graph(module_id_or_name="benefits_w18")  # full mode (small module)
```

Example response (real output)


```
{
  "module_id": "e337e498-34ba-47d2-a06f-c8f2c4e253be",
  "module_name": "benefits_w18",
  "source_format": "colectica_json",
  "scoped_to_question": null,
  "total_node_count": 6,
  "total_edge_count": 4,
  "mode": "full",
```

```
  "graph_start_summary": {
    "first_module_question_id": "AIDHH",
    "true_start_question_ids": [
      "AIDHH"
    ],
    "external_entry_prerequisites": [],
    "has_true_unconditional_start": true,
    "has_external_entry_prerequisites": false
  },
  "graph": {
    "nodes_json": [
      {
        "id": "AIDHH",
        "data": {
          "label": "AIDHH",
          "sequence_index": 1
        },
        "type": "question",
        "incoming_routes": [],
        "outgoing_routes": [
          {
            "id": "e1",
            "source": "AIDHH",
            "source_parts": [
              "AIDHH"
            ],
            "target": "NAIDHH",
            "type": "conditional",
            "label": "",
            "condition_text": "if [AIDHH = 1]"
          }
        ],
        "prerequisite_questions": [],
        "start_semantics": {
          "is_first_module_question": true,
          "is_true_unconditional_start": true,
          "is_entry_with_external_prerequisites": false,
          "has_incoming_routes": false,
          "has_external_prerequisites": false,
          "has_internal_prerequisites": false,
          "has_conditional_incoming_routes": false,
          "start_kind": "true_unconditional_start",
          "display_label": "True unconditional start",
            "explanation": "This is the first question inside the module and no incoming or external prerequisite
routes were detected."
        },
        "dependency_summary": {
          "incoming_route_count": 0,
          "outgoing_route_count": 1,
          "prerequisite_question_count": 0,
          "has_external_prerequisites": false,
          "has_conditional_incoming_routes": false
        }
      },
      {
        "id": "NAIDHH",
        "data": {
          "label": "NAIDHH",
          "sequence_index": 2
```

```
      },
      "type": "question",
      "incoming_routes": [
        {
          "id": "e1",
          "source": "AIDHH",
          "source_parts": [
            "AIDHH"
          ],
          "target": "NAIDHH",
          "type": "conditional",
          "label": "",
          "condition_text": "if [AIDHH = 1]"
        }
      ],
      "outgoing_routes": [
        {
          "id": "e2",
          "source": "NAIDHH",
          "source_parts": [
            "NAIDHH"
          ],
          "target": "BENBASE",
          "type": "conditional",
          "label": "",
          "condition_text": "if [NAIDHH > 1]"
        }
      ],
      "prerequisite_questions": [
        {
          "id": "AIDHH",
          "exists_in_graph": true,
          "is_external": false,
          "type": "question",
          "label": "AIDHH"
        }
      ],
      "start_semantics": {
        "is_first_module_question": false,
        "is_true_unconditional_start": false,
        "is_entry_with_external_prerequisites": false,
        "has_incoming_routes": true,
        "has_external_prerequisites": false,
        "has_internal_prerequisites": true,
        "has_conditional_incoming_routes": true,
        "start_kind": "not_start",
        "display_label": "Not a module start",
        "explanation": "This is not the first question detected inside this module."
      },
      "dependency_summary": {
        "incoming_route_count": 1,
        "outgoing_route_count": 1,
        "prerequisite_question_count": 1,
        "has_external_prerequisites": false,
        "has_conditional_incoming_routes": true
      }
    },
    {
      "id": "BENBASE",
```

```
        "data": {
          "label": "BENBASE",
          "sequence_index": 3
        },
        "type": "question",
        "incoming_routes": [
          {
            "id": "e2",
            "source": "NAIDHH",
            "source_parts": [
              "NAIDHH"
            ],
            "target": "BENBASE",
            "type": "conditional",
            "label": "",
            "condition_text": "if [NAIDHH > 1]"
          }
        ],
        "outgoing_routes": [],
        "prerequisite_questions": [
          {
            "id": "NAIDHH",
            "exists_in_graph": true,
            "is_external": false,
            "type": "question",
            "label": "NAIDHH"
          }
        ],
        "start_semantics": {
          "is_first_module_question": false,
          "is_true_unconditional_start": false,
          "is_entry_with_external_prerequisites": false,
          "has_incoming_routes": true,
          "has_external_prerequisites": false,
          "has_internal_prerequisites": true,
          "has_conditional_incoming_routes": true,
          "start_kind": "not_start",
          "display_label": "Not a module start",
          "explanation": "This is not the first question detected inside this module."
        },
        "dependency_summary": {
          "incoming_route_count": 1,
          "outgoing_route_count": 0,
          "prerequisite_question_count": 1,
          "has_external_prerequisites": false,
          "has_conditional_incoming_routes": true
        }
      },
      {
        "id": "FICODE",
        "data": {
          "label": "FICODE",
          "sequence_index": 4
        },
        "type": "question",
        "incoming_routes": [
          {
            "id": "e3",
            "source": "HHGRID.HHSIZE",
```

```
          "source_parts": [
            "HHGRID.HHSIZE"
          ],
          "target": "FICODE",
          "type": "conditional",
          "label": "",
          "condition_text": "if [(HHGRID.HHSIZE - absent members) > 1]"
        }
      ],
      "outgoing_routes": [
        {
          "id": "e4",
          "source": "FICODE",
          "source_parts": [
            "FICODE"
          ],
          "target": "FRVAL",
          "type": "sequential",
          "label": "",
          "condition_text": ""
        }
      ],
      "prerequisite_questions": [
        {
          "id": "HHGRID.HHSIZE",
          "exists_in_graph": true,
          "is_external": true,
          "type": "external_variable",
          "label": "HHGRID.HHSIZE"
        }
      ],
      "start_semantics": {
        "is_first_module_question": false,
        "is_true_unconditional_start": false,
        "is_entry_with_external_prerequisites": false,
        "has_incoming_routes": true,
        "has_external_prerequisites": true,
        "has_internal_prerequisites": false,
        "has_conditional_incoming_routes": true,
        "start_kind": "not_start",
        "display_label": "Not a module start",
        "explanation": "This is not the first question detected inside this module."
      },
      "dependency_summary": {
        "incoming_route_count": 1,
        "outgoing_route_count": 1,
        "prerequisite_question_count": 1,
        "has_external_prerequisites": true,
        "has_conditional_incoming_routes": true
      }
    },
    {
      "id": "FRVAL",
      "data": {
        "label": "FRVAL",
        "sequence_index": 5
      },
      "type": "question",
      "incoming_routes": [
```

```
        {
          "id": "e4",
          "source": "FICODE",
          "source_parts": [
            "FICODE"
          ],
          "target": "FRVAL",
          "type": "sequential",
          "label": "",
          "condition_text": ""
        }
      ],
      "outgoing_routes": [],
      "prerequisite_questions": [
        {
          "id": "FICODE",
          "exists_in_graph": true,
          "is_external": false,
          "type": "question",
          "label": "FICODE"
        }
      ],
      "start_semantics": {
        "is_first_module_question": false,
        "is_true_unconditional_start": false,
        "is_entry_with_external_prerequisites": false,
        "has_incoming_routes": true,
        "has_external_prerequisites": false,
        "has_internal_prerequisites": true,
        "has_conditional_incoming_routes": false,
        "start_kind": "not_start",
        "display_label": "Not a module start",
        "explanation": "This is not the first question detected inside this module."
      },
      "dependency_summary": {
        "incoming_route_count": 1,
        "outgoing_route_count": 0,
        "prerequisite_question_count": 1,
        "has_external_prerequisites": false,
        "has_conditional_incoming_routes": false
      }
    },
    {
      "id": "HHGRID.HHSIZE",
      "data": {},
      "type": "external_variable"
    }
  ],
  "edges_json": [
    {
      "id": "e1",
      "type": "conditional",
      "source": "AIDHH",
      "target": "NAIDHH",
      "condition_text": "if [AIDHH = 1]"
    },
    {
      "id": "e2",
      "type": "conditional",
```

```
        "source": "NAIDHH",
        "target": "BENBASE",
        "condition_text": "if [NAIDHH > 1]"
      },
      {
        "id": "e3",
        "type": "conditional",
        "source": "HHGRID.HHSIZE",
        "target": "FICODE",
        "condition_text": "if [(HHGRID.HHSIZE - absent members) > 1]"
      },
      {
        "id": "e4",
        "type": "sequential",
        "source": "FICODE",
        "target": "FRVAL"
      }
    ],
    "graph_start_summary": {
      "first_module_question_id": "AIDHH",
      "true_start_question_ids": [
        "AIDHH"
      ],
      "external_entry_prerequisites": [],
      "has_true_unconditional_start": true,
      "has_external_entry_prerequisites": false
    }
  },
   "mermaid": "flowchart TD\n      AIDHH[\"AIDHH\"]\n      NAIDHH[\"NAIDHH\"]\n      BENBASE[\"BENBASE\"]\n
FICODE[\"FICODE\"]\n     FRVAL[\"FRVAL\"]\n     HHGRID_HHSIZE((\"HHGRID.HHSIZE\"))\n     AIDHH -->|\"conditional\"|
NAIDHH\n      NAIDHH -->|\"conditional\"| BENBASE\n      HHGRID_HHSIZE -->|\"conditional\"| FICODE\n      FICODE
-->|\"sequential\"| FRVAL"
}
```


Second example: neighborhood mode

```
get_module_graph(module_id_or_name="benefits_w18", question_name="NAIDHH", depth=1)    # neighborhood
mode
```

Example response (real output)


```
{
  "module_id": "e337e498-34ba-47d2-a06f-c8f2c4e253be",
  "module_name": "benefits_w18",
  "source_format": "colectica_json",
  "scoped_to_question": "NAIDHH",
  "total_node_count": 6,
  "total_edge_count": 4,
  "mode": "neighborhood",
  "graph_start_summary": {
    "first_module_question_id": "AIDHH",
    "true_start_question_ids": [
      "AIDHH"
    ],
    "external_entry_prerequisites": [],
    "has_true_unconditional_start": true,
    "has_external_entry_prerequisites": false
  },
  "graph": {
    "nodes_json": [
      {
```

```
        "id": "AIDHH",
        "data": {
          "label": "AIDHH",
          "sequence_index": 1
        },
        "type": "question",
        "incoming_routes": [],
        "outgoing_routes": [
          {
            "id": "e1",
            "source": "AIDHH",
            "source_parts": [
              "AIDHH"
            ],
            "target": "NAIDHH",
            "type": "conditional",
            "label": "",
            "condition_text": "if [AIDHH = 1]"
          }
        ],
        "prerequisite_questions": [],
        "start_semantics": {
          "is_first_module_question": true,
          "is_true_unconditional_start": true,
          "is_entry_with_external_prerequisites": false,
          "has_incoming_routes": false,
          "has_external_prerequisites": false,
          "has_internal_prerequisites": false,
          "has_conditional_incoming_routes": false,
          "start_kind": "true_unconditional_start",
          "display_label": "True unconditional start",
            "explanation": "This is the first question inside the module and no incoming or external prerequisite
routes were detected."
        },
        "dependency_summary": {
          "incoming_route_count": 0,
          "outgoing_route_count": 1,
          "prerequisite_question_count": 0,
          "has_external_prerequisites": false,
          "has_conditional_incoming_routes": false
        }
      },
      {
        "id": "NAIDHH",
        "data": {
          "label": "NAIDHH",
          "sequence_index": 2
        },
        "type": "question",
        "incoming_routes": [
          {
            "id": "e1",
            "source": "AIDHH",
            "source_parts": [
              "AIDHH"
            ],
            "target": "NAIDHH",
            "type": "conditional",
            "label": "",
```

```
            "condition_text": "if [AIDHH = 1]"
          }
        ],
        "outgoing_routes": [
          {
            "id": "e2",
            "source": "NAIDHH",
            "source_parts": [
              "NAIDHH"
            ],
            "target": "BENBASE",
            "type": "conditional",
            "label": "",
            "condition_text": "if [NAIDHH > 1]"
          }
        ],
        "prerequisite_questions": [
          {
            "id": "AIDHH",
            "exists_in_graph": true,
            "is_external": false,
            "type": "question",
            "label": "AIDHH"
          }
        ],
        "start_semantics": {
          "is_first_module_question": false,
          "is_true_unconditional_start": false,
          "is_entry_with_external_prerequisites": false,
          "has_incoming_routes": true,
          "has_external_prerequisites": false,
          "has_internal_prerequisites": true,
          "has_conditional_incoming_routes": true,
          "start_kind": "not_start",
          "display_label": "Not a module start",
          "explanation": "This is not the first question detected inside this module."
        },
        "dependency_summary": {
          "incoming_route_count": 1,
          "outgoing_route_count": 1,
          "prerequisite_question_count": 1,
          "has_external_prerequisites": false,
          "has_conditional_incoming_routes": true
        }
      },
      {
        "id": "BENBASE",
        "data": {
          "label": "BENBASE",
          "sequence_index": 3
        },
        "type": "question",
        "incoming_routes": [
          {
            "id": "e2",
            "source": "NAIDHH",
            "source_parts": [
              "NAIDHH"
            ],
```

```
          "target": "BENBASE",
          "type": "conditional",
          "label": "",
          "condition_text": "if [NAIDHH > 1]"
        }
      ],
      "outgoing_routes": [],
      "prerequisite_questions": [
        {
          "id": "NAIDHH",
          "exists_in_graph": true,
          "is_external": false,
          "type": "question",
          "label": "NAIDHH"
        }
      ],
      "start_semantics": {
        "is_first_module_question": false,
        "is_true_unconditional_start": false,
        "is_entry_with_external_prerequisites": false,
        "has_incoming_routes": true,
        "has_external_prerequisites": false,
        "has_internal_prerequisites": true,
        "has_conditional_incoming_routes": true,
        "start_kind": "not_start",
        "display_label": "Not a module start",
        "explanation": "This is not the first question detected inside this module."
      },
      "dependency_summary": {
        "incoming_route_count": 1,
        "outgoing_route_count": 0,
        "prerequisite_question_count": 1,
        "has_external_prerequisites": false,
        "has_conditional_incoming_routes": true
      }
    }
  ],
  "edges_json": [
    {
      "id": "e1",
      "type": "conditional",
      "source": "AIDHH",
      "target": "NAIDHH",
      "condition_text": "if [AIDHH = 1]"
    },
    {
      "id": "e2",
      "type": "conditional",
      "source": "NAIDHH",
      "target": "BENBASE",
      "condition_text": "if [NAIDHH > 1]"
    }
  ],
  "graph_start_summary": {
    "first_module_question_id": "AIDHH",
    "true_start_question_ids": [
      "AIDHH"
    ],
    "external_entry_prerequisites": [],
```

```
      "has_true_unconditional_start": true,
      "has_external_entry_prerequisites": false
    }
  },
   "mermaid": "flowchart TD\n     AIDHH[\"AIDHH\"]\n     NAIDHH[\"NAIDHH\"]\n     BENBASE[\"BENBASE\"]\n     AIDHH
-->|\"conditional\"| NAIDHH\n    NAIDHH -->|\"conditional\"| BENBASE"
}
```

evaluate_edge_condition

Scope: Colectica + Forsta+

MCP prompt: /evaluateCondition

Description

Evaluate a hypothetical condition expression against hypothetical answers -- Colectica OR Forsta+, both supported (the grammar picked automatically from the module's source_format). condition_text does not need to belong to any real RoutingEdge in this module; this lets a client ask "what if" questions interactively (e.g. would this fire if AIDHH=1), not just replay edges that already exist.

Pure/side-effect-free. Returns status ("true"/"false"/"unknown"/"unsupported"), missing_inputs, used_inputs, normalized_expression, and warnings -- unchanged from the underlying evaluator.

Python signature

```
evaluate_edge_condition(
    module_id_or_name: str,
    condition_text: str,
    inputs: dict[str, Any]
) -> dict
```

Example prompt shortcut

```
/evaluateCondition

"Using the evaluate_edge_condition tool, evaluate the condition 'if [AIDHH = 1]' for module 'benefits_w18' against these hypothetical answers: {'AIDHH': '1'}. Tell me whether it fires (true/false/unknown/unsupported) and which inputs were used or missing."
```

Example request

```
evaluate_edge_condition(module_id_or_name="benefits_w18", condition_text="if [AIDHH = 1]", inputs={"AIDHH": "1"})
```

Example response (real output)


```
{
  "module_id": "e337e498-34ba-47d2-a06f-c8f2c4e253be",
  "module_name": "benefits_w18",
  "source_format": "colectica_json",
  "condition_text": "if [AIDHH = 1]",
  "status": "true",
  "missing_inputs": [],
  "used_inputs": {
    "AIDHH": "1"
  },
  "normalized_expression": "AIDHH = 1",
  "warnings": []
}
```

COLECTICA VS FORSTA+ ROUTING DIFF

get_routing_diff_report

Scope: Colectica + Forsta+ pair

MCP prompt: /showRoutingDiffReport

Description

Structural routing discrepancies between a matched Colectica/Forsta+ module pair, exactly matching the routing-diff report GUI page (/questionnaires/routing-diff/<colectica_id>/<forsta_id>/). Does NOT run matching/comparison itself (build_question_matches/ compare_routing_for_modules are expensive for large modules) -- if matching has never been run for this exact pair, has_run is False and discrepancies is empty rather than raising.

Each discrepancy is a *lead for manual review*, not a confirmed bug -- see each row's "meaning" field: this is a structural comparison only, it never evaluates what a condition means.

discrepancy_type filters to one DiscrepancyType value. limit caps how many discrepancies are returned (default 200) -- discrepancy_count always reports the true total regardless of limit, since large modules can have hundreds of rows with full explainer text each.

Python signature

```
get_routing_diff_report(
    colectica_module_id_or_name: str,
    forsta_module_id_or_name: str,
    discrepancy_type: str | None = None,
    limit: int = 200
) -> dict
```

Example prompt shortcut

```
/showRoutingDiffReport

"Using the get_routing_diff_report tool, show me the routing-diff report between Colectica module 'benefits_w18' and Forsta+ module 'benefits_w18_forsta'. If has_run is false, tell me matching/comparison hasn't been run for this pair yet instead of guessing. Otherwise summarize the match_summary and group the discrepancies by type and severity."
```

Example request

```
get_routing_diff_report(colectica_module_id_or_name="benefits_w18",
forsta_module_id_or_name="benefits_w18_forsta")
```

Example response (real output)


```
{
  "colectica_module_id": "e337e498-34ba-47d2-a06f-c8f2c4e253be",
  "colectica_module_name": "benefits_w18",
  "forsta_module_id": "5ed9feaa-e746-4671-a7c1-06d4279b6b19",
  "forsta_module_name": "benefits_w18_forsta",
  "has_run": true,
  "match_summary": {
    "total": 5,
    "matched": 1,
    "unmatched": 4
```

```
  },
  "discrepancy_count": 1,
  "discrepancies": [
    {
      "id": "d7551b9f-7bbd-47be-83a9-9df334dd20b1",
      "discrepancy_type": "missing_in_forsta",
      "severity": "warning",
      "colectica_question": "AIDHH",
      "forsta_question": "AIDHH",
       "summary": "Question AIDHH matched in both systems, but the routing edge to BENBASE (fires when NAIDHH > 1) has no equivalent in Forsta+.",
       "meaning": "This is a structural comparison only: it checks whether an edge with the same source and target exists on both sides, not what the condition means or whether an alternate path exists. This flag means either (a) Forsta+ genuinely never routes to BENBASE from here -- a real gap worth fixing, or (b) Forsta+ reaches BENBASE some other way (a different source question, different condition logic, a default fallthrough) that this structural check can't recognize -- a false positive. Severity \"Warning\" means this needs a human to check, not that it's a confirmed bug.",
      "edge": {
        "origin_system": "Colectica",
        "source_question": "NAIDHH",
        "target_question": "BENBASE",
        "condition_text": "NAIDHH > 1"
      }
    }
  ]
}
```

get_routing_discrepancy_detail

Scope: Colectica + Forsta+ pair

MCP prompt: /showColecticaForstaDiscrepancy

Description

One routing discrepancy in full detail, exactly matching the routing-diff
discrepancy detail GUI page
(/questionnaires/routing-diff/<colectica_id>/<forsta_id>/discrepancy/<discrepancy_id>/)
-- including BOTH systems' routing graph neighborhoods around the focus
question (colectica_graph and forsta_graph), not just Colectica's side.
Either graph is None if that module has no graph built yet, rather than
erroring the whole call.

Python signature

```
get_routing_discrepancy_detail(
    colectica_module_id_or_name: str,
    forsta_module_id_or_name: str,
    discrepancy_id: str
) -> dict
```

Example prompt shortcut

```
/showColecticaForstaDiscrepancy

"Using the get_routing_discrepancy_detail tool, show me full detail for discrepancy 'd7551b9f-7bbd-47be-83a9-9df334dd20b1' between Colectica module 'benefits_w18' and Forsta+ module 'benefits_w18_forsta' -- the summary, the meaning (why this is a lead not a confirmed bug), and describe the routing around the focus question on both the Colectica graph and the Forsta+ graph."
```

Example request

```
get_routing_discrepancy_detail(colectica_module_id_or_name="benefits_w18",
forsta_module_id_or_name="benefits_w18_forsta", discrepancy_id="d7551b9f-7bbd-47be-83a9-9df334dd20b1")
```

Example response (real output)


```
{
  "colectica_module_id": "e337e498-34ba-47d2-a06f-c8f2c4e253be",
  "forsta_module_id": "5ed9feaa-e746-4671-a7c1-06d4279b6b19",
  "discrepancy_id": "d7551b9f-7bbd-47be-83a9-9df334dd20b1",
  "discrepancy_type": "missing_in_forsta",
  "severity": "warning",
  "summary": "Question AIDHH matched in both systems, but the routing edge to BENBASE (fires when NAIDHH > 1) has no equivalent in Forsta+.",
   "meaning": "This is a structural comparison only: it checks whether an edge with the same source and target exists on both sides, not what the condition means or whether an alternate path exists. This flag means either (a) Forsta+ genuinely never routes to BENBASE from here -- a real gap worth fixing, or (b) Forsta+ reaches BENBASE some other way (a different source question, different condition logic, a default fallthrough) that this structural check can't recognize -- a false positive. Severity \"Warning\" means this needs a human to check, not that it's a confirmed bug.",
  "edge": {
    "origin_system": "Colectica",
    "source_question": "NAIDHH",
    "target_question": "BENBASE",
    "condition_text": "NAIDHH > 1"
  },
  "focus_question_name": "AIDHH",
  "colectica_focus_question_name": "AIDHH",
  "forsta_focus_question_name": "AIDHH",
```

```
  "highlight_edge": {
    "source": "NAIDHH",
    "target": "BENBASE"
  },
  "colectica_graph": {
    "nodes_json": [
      {
        "id": "AIDHH",
        "data": {
          "label": "AIDHH",
          "sequence_index": 1
        },
        "type": "question"
      },
      {
        "id": "NAIDHH",
        "data": {
          "label": "NAIDHH",
          "sequence_index": 2
        },
        "type": "question"
      }
    ],
    "edges_json": [
      {
        "id": "e1",
        "type": "conditional",
        "source": "AIDHH",
        "target": "NAIDHH",
        "condition_text": "if [AIDHH = 1]"
      }
    ]
  },
  "forsta_graph": {
    "nodes_json": [
      {
        "id": "AIDHH",
        "data": {
          "label": "AIDHH",
          "sequence_index": 1
        },
        "type": "question"
      },
      {
        "id": "BENBASE",
        "data": {
          "label": "BENBASE",
          "sequence_index": 2
        },
        "type": "question"
      }
    ],
    "edges_json": [
      {
        "id": "e1",
        "type": "conditional",
        "source": "AIDHH",
        "target": "BENBASE",
        "condition_text": "f('AIDHH').any('1')"
```

```
      }
    ]
  }
}
```

get_routing_simulation

Scope: Colectica-only

MCP prompt: /showRoutingSimulation

Description

Deterministic coverage-intent simulation for a Colectica module: for each of up to 10 auto-generated coverage intents, which conditional edges fired given seed answers, and whether each intent's target edge(s) were covered. Exactly replicates what the routing-simulation GUI JSON endpoint already computes -- not persisted, recomputed on every call, same cost profile as that existing endpoint. Bounded regardless of module size (coverage_intent_builder caps at 10 intents).

Python signature

```
get_routing_simulation(
    module_id_or_name: str
) -> dict
```

Example prompt shortcut

```
/showRoutingSimulation

"Using the get_routing_simulation tool, show me the routing coverage simulation for module 'benefits_w18'. Summarize status_counts and call out any intents that are not_covered or blocked_missing_inputs."
```

Example request

```
get_routing_simulation(module_id_or_name="benefits_w18")
```

Example response (real output)


```
{
  "module_id": "e337e498-34ba-47d2-a06f-c8f2c4e253be",
  "module_name": "benefits_w18",
  "coverage_intent_count": 3,
  "status_counts": {
    "covered": 2,
    "unknown": 1
  },
  "results": [
    {
      "intent_id": "Trigger NAIDHH",
      "intent_label": "Trigger NAIDHH",
      "inputs": {
        "AIDHH": "1"
      },
      "active_questions": [],
      "skipped_questions": [],
      "triggered_edges": [
        {
          "edge_index": 2,
          "edge_id": "efa05fd1-e826-40ad-942f-1a1788b46788",
          "source": "AIDHH",
          "target": "NAIDHH",
          "condition_text": "if [AIDHH = 1]",
          "evaluation_status": "true",
          "used_inputs": {
            "AIDHH": "1"
```

```
      },
      "missing_inputs": [],
      "normalized_expression": "AIDHH = 1",
      "warnings": [],
      "is_target_edge": true
    }
  ],
  "untriggered_edges": [
    {
      "edge_index": 0,
      "edge_id": "3a07b29c-aaea-4102-a69d-87eaf2fe6a01",
      "source": "HHGRID.HHSIZE",
      "target": "FICODE",
      "condition_text": "if [(HHGRID.HHSIZE - absent members) > 1]",
      "evaluation_status": "unsupported",
      "used_inputs": {},
      "missing_inputs": [],
      "normalized_expression": "(HHGRID.HHSIZE - absent members) > 1",
      "warnings": [
        "Unsupported atomic condition: (HHGRID.HHSIZE - absent members) > 1"
      ],
      "is_target_edge": false
    },
    {
      "edge_index": 1,
      "edge_id": "6b69de97-bdf8-4ce6-b2f5-d2ab195499d6",
      "source": "NAIDHH",
      "target": "BENBASE",
      "condition_text": "if [NAIDHH > 1]",
      "evaluation_status": "unknown",
      "used_inputs": {},
      "missing_inputs": [
        "NAIDHH"
      ],
      "normalized_expression": "NAIDHH > 1",
      "warnings": [],
      "is_target_edge": false
    }
  ],
  "target_edges": [
    {
      "edge_index": 2,
      "edge_id": "efa05fd1-e826-40ad-942f-1a1788b46788",
      "source": "AIDHH",
      "target": "NAIDHH",
      "condition_text": "if [AIDHH = 1]",
      "evaluation_status": "true",
      "used_inputs": {
        "AIDHH": "1"
      },
      "missing_inputs": [],
      "normalized_expression": "AIDHH = 1",
      "warnings": [],
      "is_target_edge": true
    }
  ],
  "non_target_edges": [
    {
      "edge_index": 0,
```

```
          "edge_id": "3a07b29c-aaea-4102-a69d-87eaf2fe6a01",
          "source": "HHGRID.HHSIZE",
          "target": "FICODE",
          "condition_text": "if [(HHGRID.HHSIZE - absent members) > 1]",
          "evaluation_status": "unsupported",
          "used_inputs": {},
          "missing_inputs": [],
          "normalized_expression": "(HHGRID.HHSIZE - absent members) > 1",
          "warnings": [
            "Unsupported atomic condition: (HHGRID.HHSIZE - absent members) > 1"
          ],
          "is_target_edge": false
        },
        {
          "edge_index": 1,
          "edge_id": "6b69de97-bdf8-4ce6-b2f5-d2ab195499d6",
          "source": "NAIDHH",
          "target": "BENBASE",
          "condition_text": "if [NAIDHH > 1]",
          "evaluation_status": "unknown",
          "used_inputs": {},
          "missing_inputs": [
            "NAIDHH"
          ],
          "normalized_expression": "NAIDHH > 1",
          "warnings": [],
          "is_target_edge": false
        }
      ],
      "coverage_status": "covered",
      "coverage_summary": {
        "declared_target_edge_indexes": [
          0
        ],
        "matched_target_edge_indexes": [
          2
        ],
        "target_edge_count": 1,
        "triggered_target_edge_count": 1,
        "false_target_edge_count": 0,
        "missing_target_edge_count": 0,
        "unknown_target_edge_count": 0,
        "triggered_target_edge_indexes": [
          2
        ],
        "missing_target_edge_indexes": [],
        "false_target_edge_indexes": []
      },
      "missing_inputs": [
        "NAIDHH"
      ],
      "invalid_inputs": [],
      "warnings": [
        "Edge index 0 (3a07b29c-aaea-4102-a69d-87eaf2fe6a01): Unsupported atomic condition: (HHGRID.HHSIZE - absent
members) > 1",
        "Edge index 0 (3a07b29c-aaea-4102-a69d-87eaf2fe6a01): unsupported evaluator status 'unsupported'.",
        "Edge index 1 (6b69de97-bdf8-4ce6-b2f5-d2ab195499d6): unsupported evaluator status 'unknown'.",
            "Step 16.4 evaluates all conditional edges globally and separately reports target edge coverage.
Active/skipped question traversal is not implemented yet."
```

```
      ]
    },
    {
      "intent_id": "Trigger BENBASE",
      "intent_label": "Trigger BENBASE",
      "inputs": {
        "NAIDHH": 2,
        "AIDHH": "1"
      },
      "active_questions": [],
      "skipped_questions": [],
      "triggered_edges": [
        {
          "edge_index": 1,
          "edge_id": "6b69de97-bdf8-4ce6-b2f5-d2ab195499d6",
          "source": "NAIDHH",
          "target": "BENBASE",
          "condition_text": "if [NAIDHH > 1]",
          "evaluation_status": "true",
          "used_inputs": {
            "NAIDHH": 2
          },
          "missing_inputs": [],
          "normalized_expression": "NAIDHH > 1",
          "warnings": [],
          "is_target_edge": true
        },
        {
          "edge_index": 2,
          "edge_id": "efa05fd1-e826-40ad-942f-1a1788b46788",
          "source": "AIDHH",
          "target": "NAIDHH",
          "condition_text": "if [AIDHH = 1]",
          "evaluation_status": "true",
          "used_inputs": {
            "AIDHH": "1"
          },
          "missing_inputs": [],
          "normalized_expression": "AIDHH = 1",
          "warnings": [],
          "is_target_edge": false
        }
      ],
      "untriggered_edges": [
        {
          "edge_index": 0,
          "edge_id": "3a07b29c-aaea-4102-a69d-87eaf2fe6a01",
          "source": "HHGRID.HHSIZE",
          "target": "FICODE",
          "condition_text": "if [(HHGRID.HHSIZE - absent members) > 1]",
          "evaluation_status": "unsupported",
          "used_inputs": {},
          "missing_inputs": [],
          "normalized_expression": "(HHGRID.HHSIZE - absent members) > 1",
          "warnings": [
            "Unsupported atomic condition: (HHGRID.HHSIZE - absent members) > 1"
          ],
          "is_target_edge": false
        }
```

```
    ],
    "target_edges": [
      {
        "edge_index": 1,
        "edge_id": "6b69de97-bdf8-4ce6-b2f5-d2ab195499d6",
        "source": "NAIDHH",
        "target": "BENBASE",
        "condition_text": "if [NAIDHH > 1]",
        "evaluation_status": "true",
        "used_inputs": {
          "NAIDHH": 2
        },
        "missing_inputs": [],
        "normalized_expression": "NAIDHH > 1",
        "warnings": [],
        "is_target_edge": true
      }
    ],
    "non_target_edges": [
      {
        "edge_index": 0,
        "edge_id": "3a07b29c-aaea-4102-a69d-87eaf2fe6a01",
        "source": "HHGRID.HHSIZE",
        "target": "FICODE",
        "condition_text": "if [(HHGRID.HHSIZE - absent members) > 1]",
        "evaluation_status": "unsupported",
        "used_inputs": {},
        "missing_inputs": [],
        "normalized_expression": "(HHGRID.HHSIZE - absent members) > 1",
        "warnings": [
          "Unsupported atomic condition: (HHGRID.HHSIZE - absent members) > 1"
        ],
        "is_target_edge": false
      },
      {
        "edge_index": 2,
        "edge_id": "efa05fd1-e826-40ad-942f-1a1788b46788",
        "source": "AIDHH",
        "target": "NAIDHH",
        "condition_text": "if [AIDHH = 1]",
        "evaluation_status": "true",
        "used_inputs": {
          "AIDHH": "1"
        },
        "missing_inputs": [],
        "normalized_expression": "AIDHH = 1",
        "warnings": [],
        "is_target_edge": false
      }
    ],
    "coverage_status": "covered",
    "coverage_summary": {
      "declared_target_edge_indexes": [
        1
      ],
      "matched_target_edge_indexes": [
        1
      ],
      "target_edge_count": 1,
```

```
        "triggered_target_edge_count": 1,
        "false_target_edge_count": 0,
        "missing_target_edge_count": 0,
        "unknown_target_edge_count": 0,
        "triggered_target_edge_indexes": [
          1
        ],
        "missing_target_edge_indexes": [],
        "false_target_edge_indexes": []
      },
      "missing_inputs": [],
      "invalid_inputs": [],
      "warnings": [
        "Edge index 0 (3a07b29c-aaea-4102-a69d-87eaf2fe6a01): Unsupported atomic condition: (HHGRID.HHSIZE - absent
members) > 1",
        "Edge index 0 (3a07b29c-aaea-4102-a69d-87eaf2fe6a01): unsupported evaluator status 'unsupported'.",
            "Step 16.4 evaluates all conditional edges globally and separately reports target edge coverage.
Active/skipped question traversal is not implemented yet."
      ]
    },
    {
      "intent_id": "Trigger FICODE",
      "intent_label": "Trigger FICODE",
      "inputs": {
        "HHGRID.HHSIZE": "2"
      },
      "active_questions": [],
      "skipped_questions": [],
      "triggered_edges": [],
      "untriggered_edges": [
        {
          "edge_index": 0,
          "edge_id": "3a07b29c-aaea-4102-a69d-87eaf2fe6a01",
          "source": "HHGRID.HHSIZE",
          "target": "FICODE",
          "condition_text": "if [(HHGRID.HHSIZE - absent members) > 1]",
          "evaluation_status": "unsupported",
          "used_inputs": {},
          "missing_inputs": [],
          "normalized_expression": "(HHGRID.HHSIZE - absent members) > 1",
          "warnings": [
            "Unsupported atomic condition: (HHGRID.HHSIZE - absent members) > 1"
          ],
          "is_target_edge": true
        },
        {
          "edge_index": 1,
          "edge_id": "6b69de97-bdf8-4ce6-b2f5-d2ab195499d6",
          "source": "NAIDHH",
          "target": "BENBASE",
          "condition_text": "if [NAIDHH > 1]",
          "evaluation_status": "unknown",
          "used_inputs": {},
          "missing_inputs": [
            "NAIDHH"
          ],
          "normalized_expression": "NAIDHH > 1",
          "warnings": [],
          "is_target_edge": false
```

```
    },
    {
      "edge_index": 2,
      "edge_id": "efa05fd1-e826-40ad-942f-1a1788b46788",
      "source": "AIDHH",
      "target": "NAIDHH",
      "condition_text": "if [AIDHH = 1]",
      "evaluation_status": "unknown",
      "used_inputs": {},
      "missing_inputs": [
        "AIDHH"
      ],
      "normalized_expression": "AIDHH = 1",
      "warnings": [],
      "is_target_edge": false
    }
  ],
  "target_edges": [
    {
      "edge_index": 0,
      "edge_id": "3a07b29c-aaea-4102-a69d-87eaf2fe6a01",
      "source": "HHGRID.HHSIZE",
      "target": "FICODE",
      "condition_text": "if [(HHGRID.HHSIZE - absent members) > 1]",
      "evaluation_status": "unsupported",
      "used_inputs": {},
      "missing_inputs": [],
      "normalized_expression": "(HHGRID.HHSIZE - absent members) > 1",
      "warnings": [
        "Unsupported atomic condition: (HHGRID.HHSIZE - absent members) > 1"
      ],
      "is_target_edge": true
    }
  ],
  "non_target_edges": [
    {
      "edge_index": 1,
      "edge_id": "6b69de97-bdf8-4ce6-b2f5-d2ab195499d6",
      "source": "NAIDHH",
      "target": "BENBASE",
      "condition_text": "if [NAIDHH > 1]",
      "evaluation_status": "unknown",
      "used_inputs": {},
      "missing_inputs": [
        "NAIDHH"
      ],
      "normalized_expression": "NAIDHH > 1",
      "warnings": [],
      "is_target_edge": false
    },
    {
      "edge_index": 2,
      "edge_id": "efa05fd1-e826-40ad-942f-1a1788b46788",
      "source": "AIDHH",
      "target": "NAIDHH",
      "condition_text": "if [AIDHH = 1]",
      "evaluation_status": "unknown",
      "used_inputs": {},
      "missing_inputs": [
```

```
            "AIDHH"
          ],
          "normalized_expression": "AIDHH = 1",
          "warnings": [],
          "is_target_edge": false
        }
      ],
      "coverage_status": "unknown",
      "coverage_summary": {
        "declared_target_edge_indexes": [
          2
        ],
        "matched_target_edge_indexes": [
          0
        ],
        "target_edge_count": 1,
        "triggered_target_edge_count": 0,
        "false_target_edge_count": 0,
        "missing_target_edge_count": 0,
        "unknown_target_edge_count": 1,
        "triggered_target_edge_indexes": [],
        "missing_target_edge_indexes": [],
        "false_target_edge_indexes": []
      },
      "missing_inputs": [
        "AIDHH",
        "NAIDHH"
      ],
      "invalid_inputs": [],
      "warnings": [
        "Edge index 0 (3a07b29c-aaea-4102-a69d-87eaf2fe6a01): Unsupported atomic condition: (HHGRID.HHSIZE - absent
members) > 1",
        "Edge index 0 (3a07b29c-aaea-4102-a69d-87eaf2fe6a01): unsupported evaluator status 'unsupported'.",
        "Edge index 1 (6b69de97-bdf8-4ce6-b2f5-d2ab195499d6): unsupported evaluator status 'unknown'.",
        "Edge index 2 (efa05fd1-e826-40ad-942f-1a1788b46788): unsupported evaluator status 'unknown'.",
            "Step 16.4 evaluates all conditional edges globally and separately reports target edge coverage.
Active/skipped question traversal is not implemented yet."
      ]
    }
  ]
}
```

COLECTICA VS COLECTICA ROUTING DIFF (VERSION DIFF)

get_version_diff_report

Scope: Colectica + Colectica pair

MCP prompt: /showVersionDiffReport

Description

Structural (plus condition-text) routing discrepancies between two Colectica-format modules -- e.g. two waves/versions of the same questionnaire -- exactly matching the version-diff report GUI page (/questionnaires/version-diff/<base_id>/<compare_id>/). Does NOT run matching/comparison itself -- if it's never been run for this exact pair, has_run is False and discrepancies is empty rather than raising.

Both modules must be Colectica JSON; a Forsta+ module raises ToolError. This is a separate pipeline from get_routing_diff_report (Colectica vs Forsta+) -- different matcher priority (name-first) and comparator (adds a CONDITION_MISMATCH type, meaningful here specifically because both sides share the same Colectica grammar).

discrepancy_type filters to one VersionRoutingDiscrepancy.DiscrepancyType value (missing_in_compare / missing_in_base / condition_mismatch). limit caps how many discrepancies are returned (default 200) -- discrepancy_count always reports the true total regardless of limit.

Python signature

```
get_version_diff_report(
    base_module_id_or_name: str,
    compare_module_id_or_name: str,
    discrepancy_type: str | None = None,
    limit: int = 200
) -> dict
```

Example prompt shortcut

```
/showVersionDiffReport

"Using the get_version_diff_report tool, show me the version-diff report between base module 'benefits_w18' and compare module 'benefits_w19' (e.g. two waves/versions of the same questionnaire). If has_run is false, tell me matching/comparison hasn't been run for this pair yet instead of guessing. Otherwise summarize the match_summary and group the discrepancies by type and severity, calling out condition_mismatch rows separately since those mean both modules route to the same place but the gating condition changed."
```

Example request

```
get_version_diff_report(base_module_id_or_name="benefits_w18",
compare_module_id_or_name="benefits_w19")
```

Example response (real output)


```
{
  "base_module_id": "e337e498-34ba-47d2-a06f-c8f2c4e253be",
  "base_module_name": "benefits_w18",
  "compare_module_id": "b874d0c0-4731-4502-887d-02dfa7c70a30",
  "compare_module_name": "benefits_w19",
  "has_run": true,
```

```
  "match_summary": {
    "total": 5,
    "matched": 1,
    "unmatched": 4
  },
  "discrepancy_count": 1,
  "discrepancies": [
    {
      "id": "2a5b27a8-0a78-43a7-8073-34ed25db7840",
      "discrepancy_type": "condition_mismatch",
      "severity": "warning",
      "base_question": "AIDHH",
      "compare_question": "AIDHH",
       "summary": "Question AIDHH matched in both modules. Both modules route to NAIDHH, but the gating condition text differs: base fires when AIDHH = 1; compare fires when AIDHH = 1 | AIDHH = 2.",
       "meaning": "This is a textual comparison of the two condition expressions after normalizing whitespace and casing -- it does not evaluate whether the conditions are logically equivalent. Two differently-worded conditions can still mean exactly the same thing (and would still be reported here as different), just as two identically-normalized conditions aren't guaranteed to behave identically in every runtime edge case. Both modules agree that NAIDHH is a valid route from here -- what changed is *when* it fires. Treat this as a lead: worth checking whether the gating logic change was intentional (e.g. a design refinement between waves) or a routing regression.",
      "edge": {
        "origin_module": "base",
        "source_question": "AIDHH",
        "target_question": "NAIDHH",
        "condition_text": "AIDHH = 1"
      }
    }
  ]
}
```

get_version_diff_discrepancy_detail

Scope: Colectica + Colectica pair

MCP prompt: /showVersionDiffDiscrepancy

Description

One version-diff discrepancy in full detail, exactly matching the
version-diff discrepancy detail GUI page
(/questionnaires/version-diff/<base_id>/<compare_id>/discrepancy/<discrepancy_id>/)
-- including BOTH modules' routing graph neighborhoods around the focus
question (base_graph and compare_graph), not just the base side. Either
graph is None if that module has no graph built yet, rather than
erroring the whole call.

Python signature

```
get_version_diff_discrepancy_detail(
    base_module_id_or_name: str,
    compare_module_id_or_name: str,
    discrepancy_id: str
) -> dict
```

Example prompt shortcut

```
/showVersionDiffDiscrepancy

"Using the get_version_diff_discrepancy_detail tool, show me full detail for discrepancy '2a5b27a8-0a78-43a7-8073-34ed25db7840' between base module 'benefits_w18' and compare module 'benefits_w19' -- the summary, the meaning (why this is a lead not a confirmed bug, and if it's a condition_mismatch, that the comparison is textual not semantic), and describe the routing around the focus question on both the base graph and the compare graph."
```

Example request

```
get_version_diff_discrepancy_detail(base_module_id_or_name="benefits_w18",
compare_module_id_or_name="benefits_w19", discrepancy_id="2a5b27a8-0a78-43a7-8073-34ed25db7840")
```

Example response (real output)


```
{
  "base_module_id": "e337e498-34ba-47d2-a06f-c8f2c4e253be",
  "compare_module_id": "b874d0c0-4731-4502-887d-02dfa7c70a30",
  "discrepancy_id": "2a5b27a8-0a78-43a7-8073-34ed25db7840",
  "discrepancy_type": "condition_mismatch",
  "severity": "warning",
  "summary": "Question AIDHH matched in both modules. Both modules route to NAIDHH, but the gating condition text differs: base fires when AIDHH = 1; compare fires when AIDHH = 1 | AIDHH = 2.",
  "meaning": "This is a textual comparison of the two condition expressions after normalizing whitespace and casing -- it does not evaluate whether the conditions are logically equivalent. Two differently-worded conditions can still mean exactly the same thing (and would still be reported here as different), just as two identically-normalized conditions aren't guaranteed to behave identically in every runtime edge case. Both modules agree that NAIDHH is a valid route from here -- what changed is *when* it fires. Treat this as a lead: worth checking whether the gating logic change was intentional (e.g. a design refinement between waves) or a routing regression.",
  "edge": {
    "origin_module": "base",
    "source_question": "AIDHH",
    "target_question": "NAIDHH",
    "condition_text": "AIDHH = 1"
  },
  "focus_question_name": "AIDHH",
```

```
  "base_focus_question_name": "AIDHH",
  "compare_focus_question_name": "AIDHH",
  "highlight_edge": {
    "source": "AIDHH",
    "target": "NAIDHH"
  },
  "base_graph": {
    "nodes_json": [
      {
        "id": "AIDHH",
        "data": {
          "label": "AIDHH",
          "sequence_index": 1
        },
        "type": "question"
      },
      {
        "id": "NAIDHH",
        "data": {
          "label": "NAIDHH",
          "sequence_index": 2
        },
        "type": "question"
      }
    ],
    "edges_json": [
      {
        "id": "e1",
        "type": "conditional",
        "source": "AIDHH",
        "target": "NAIDHH",
        "condition_text": "if [AIDHH = 1]"
      }
    ]
  },
  "compare_graph": {
    "nodes_json": [
      {
        "id": "AIDHH",
        "data": {
          "label": "AIDHH",
          "sequence_index": 1
        },
        "type": "question"
      },
      {
        "id": "NAIDHH",
        "data": {
          "label": "NAIDHH",
          "sequence_index": 2
        },
        "type": "question"
      }
    ],
    "edges_json": [
      {
        "id": "e1",
        "type": "conditional",
        "source": "AIDHH",
```

```
        "target": "NAIDHH",
        "condition_text": "if [AIDHH = 1 | AIDHH = 2]"
      }
    ]
  }
}
```